

1.2 Interfaces: ein Vertrag für alle Daten

Am Bahnhof warten Taxis in einer Schlange: Wer zuerst ankommt, fährt zuerst. Die Stadt möchte dafür die Warteschlange aus 1.1 benutzen. Doch die kann bisher nur Patienten verwalten. Eine Kopie für Taxis (TaxiKnoten, TaxiWarteschlange) würde funktionieren, aber jede Verbesserung müsste man dann doppelt einbauen. *Website: Station 1.2, Projekt Taxistand*

Aufgabe 1) Was muss die Warteschlange über ihre Daten wissen?

Untersuche im Projekt die Methoden der Warteschlange. Kreuze an, ob die Methode die Daten nur weiterreicht oder eine Methode der Daten aufruft. Notiere diese gegebenenfalls.

Methode	reicht nur weiter	ruft eine Methode der Daten auf	welche?
add(Patient p)	☒	☐	
poll(), peek()	☒	☐	
ausgeben()	☐	☒	getName(): der Name zum Ausgeben

Folgerung: Von ihren Daten muss die Warteschlange nur verlangen, dass sie einen Namen (Schlüsselwort) nennen können. Genau das legen wir in einem Vertrag fest: im **Interface** DatenElement.

```
public interface DatenElement {
    String gibSchlüsselwort();
}

public class Taxi implements DatenElement {
    private int nummer;
    private String fahrer;
    ...
    public String gibSchlüsselwort() {
        return "Taxi " + nummer;
    }
}
```

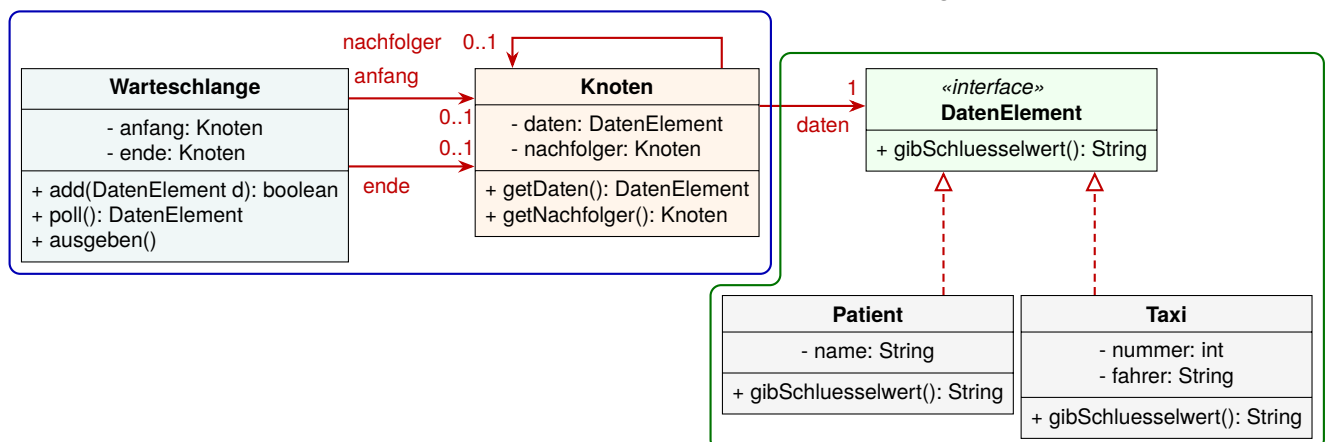
Interface

- Ein Interface ist ein **Vertrag**. Es legt nur **Methodenköpfe** fest, keine Rümpfe.
- Eine Klasse, die es implementiert (implements), muss **alle** seine Methoden bereitstellen. Das prüft der **Compiler**.
- Ein Interface ist auch ein **Datentyp**: Eine Variable vom Typ DatenElement kann Objekte jeder Klasse aufnehmen, die es implementiert. Objekte vom Interface selbst gibt es **nicht**.

Aufgabe 2) Klassendiagramm

a) Zeichne alle Beziehungen ein: Assoziationen mit Namen und Multiplizitäten, dazu implements als gestrichelten Pfeil mit leerem Dreieck zum Interface.

b) Umrande die Klassen der **Struktur** blau und die Klassen des **Inhalts** grün.



c) Durch die Trennung von Struktur und Inhalt erledigt jede Klasse genau eine Aufgabe (hohe **Kohäsion**). Struktur und Inhalt hängen nur über das Interface zusammen (lose **Kopplung**). So kann die Warteschlange ohne jede Änderung **jedes neue DatenElement** aufnehmen.

Aufgabe 3) Projekt Taxistand, TODO 3a bis 3d

Baue das Projekt so um, dass Warteschlange und Knoten mit jedem DatenElement arbeiten. Ergänze die Tabelle und übertrage die Änderungen ins Projekt (Reihenfolge wie die TODOs).

	bisher	neu
3a	DatenElement: (leer)	<code>String gibSchluesselwert();</code>
3b	<code>public class Patient {</code>	<code>public class Patient implements DatenElement {</code> <code>+ public String gibSchluesselwert() { return name; }</code>
3c	<code>private Patient daten;</code>	<code>private DatenElement daten; (ebenso Konstruktor, getDaten)</code>
3d	<code>public boolean add(Patient p)</code> <code>public Patient poll()</code> <code>aktuell.getDaten().getName()</code>	<code>public boolean add(DatenElement daten)</code> <code>public DatenElement poll() (ebenso peek)</code> <code>aktuell.getDaten().gibSchluesselwert()</code>

b) Nach dem Umbau von Knoten meldet der Compiler einen Fehler in `ausgeben()`. Erkläre!
Ein DatenElement hat nur die Methode `gibSchluesselwert()`, kein `getName()`.

Die Warteschlange darf nur benutzen, was im Vertrag steht.

Aufgabe 4) Projekt Taxistand, TODO 4

a) Ergänze den Code von Taxi oben auf Seite 1, übertrage ihn ins Projekt und kommentiere im Hauptprogramm die Taxi-Zeilen ein.

b) Lösche testweise die Methode `gibSchluesselwert()` aus Taxi. Notiere die Meldung des Compilers und erkläre sie mit dem Wort *Vertrag*.

„Die Klasse Taxi muss noch folgende Methoden des Interfaces DatenElement implementieren: ...“

Taxi verspricht mit `implements`, den Vertrag zu erfüllen, hält ihn aber nicht ein.

Aufgabe 5) Datentypen

taxistand ist eine Warteschlange. Kreuze an, welche Zeilen der Compiler akzeptiert, und begründe kurz.

Zeile	ok	Fehler	Begründung
<code>DatenElement d = new Taxi(5, "Lea");</code>	☒	☐	Taxi implementiert DatenElement.
<code>DatenElement d = new DatenElement();</code>	☐	☒	Von einem Interface gibt es keine Objekte.
<code>taxistand.add(new Patient("Ali", "Husten"));</code>	☒	☐	Auch Patient ist ein DatenElement.
<code>Taxi t = taxistand.poll();</code>	☐	☒	<code>poll()</code> liefert nur ein DatenElement.
<code>println(taxistand.poll().gibSchluesselwert());</code>	☒	☐	Diese Methode steht im Vertrag.
<code>println(taxistand.poll().getFahrer());</code>	☐	☒	<code>getFahrer()</code> steht nicht im Vertrag.

Es geht auch anders – besprecht mündlich, warum man es so machen kann:

- Jede Klasse ist ein Object. Der Knoten könnte also einfach Object daten speichern. Was ginge dann in `ausgeben()` nicht mehr?
- In einer älteren Fassung stand im Vertrag nur `String toString();`. Warum erzwingt dieser Vertrag nichts? (Tipp: Was erbt jede Klasse von Object?)
- Statt des Interfaces gäbe es eine Oberklasse Daten, von der Patient und Taxi erben. Was spricht dagegen, wenn Patient schon von Person erbt?
- Du kennst schon ein Interface: Wartezimmer aus 1.1. Was legte es fest – und wozu?